FEATURE SPEC

Analytics computations spec for the portal

Implementation spec for the analytics computations on the dev build (nikkortrinidad-cpu.github.io/flizow). Walks through every KPI, the workload bar, the capacity helpers, the drill-downs, and the rules that govern each. Includes the three audit pulls (delta chips, sparklines, over-capacity) and explains the unassigned-as-template convention so the existing rules stay aligned with that workflow.

This document is interactive. Tick each implementation step in the build checklist as you ship it, then save the PDF. State persists so we can keep one source of truth as the portal build catches up.

Overview

Defines every number Analytics renders and how each one is computed from the raw task + client + member state. The portal should mirror these rules exactly so the two builds agree on what the numbers mean. The dev build went through an honesty audit that pulled three sources of fake data (delta chips, sparklines, over-capacity KPI). Those pulls are documented below so the portal can match. A short note on the template-card workflow is included up front because it shapes how the existing rules need to behave.

Template-card workflow note

> Some managers keep an unassigned card sitting in To Do as a convenience template. They duplicate it, then assign each copy to a different teammate. We chose to support this by convention, not by adding a new field: as long as the template card stays unassigned and has no due date, every existing computation rule below will skip it correctly. No code change is needed for the template flow. The only thing the portal dev needs to do is make sure the rules below match the dev build, so the convention lands on the same invisible-to-analytics behaviour.

Filter rules

File and function

src/pages/AnalyticsPage.tsx -> applyAnalyticsFilters.

What it does

Takes the full task list and returns the subset that matches the page's current filter pills (Anyone / All projects / date window). Every computation downstream reads from this filtered list, so the rules here are upstream of every KPI and the Workload bar.

Rules

1. Drop archived tasks unconditionally (t.archived === true).

2. If filters.assigneeId is set, keep only tasks where t.assigneeId equals that id.

3. If filters.serviceId is set, keep only tasks where t.serviceId equals that id.

4. Date window: when not 'all', include tasks WITHOUT a due date (still open WIP) PLUS tasks where dueDate falls in [today - 14, today + N]. The 14-day lookback keeps recent overdues visible so the overdue counters still register them. N is 7 / 30 / 90 depending on the pill.

Team workload

File and function

src/pages/AnalyticsPage.tsx -> buildWorkload.

What it shows

One row per team member. Bar length is the share of open work that member owns relative to the busiest teammate. Number on the right is the raw open task count for that member.

Rules

1. Initialise a row for every member, wip = 0.

2. For each task: skip if !isOpen(t) (i.e. columnId === 'done'). Skip if !t.assigneeId. Skip if assignee isn't in the member list (orphan ownership). Otherwise increment that member's wip by 1.

3. After the loop, compute sharePct per row: Math.round((wip / maxWip) * 100), where maxWip = Math.max(1, ...allWips). The clamp to 1 prevents divide-by-zero when the team has zero open tasks.

4. Sort descending by wip. Members with zero open tasks land at the bottom in insertion order.

Multi-owner exclusion

Tasks with assigneeIds[] (co-owners) DO NOT contribute to any member's wip. Only the primary assigneeId field counts. The mental model is one owner per task; co-owners are participants, not capacity holders. If two designers are sharing a job, the AM splits it into two tasks.

What was pulled

An older version accrued a fake 4 hours per card toward a hardcoded 40-hour week and rendered a percent-of-capacity bar in front of the count. The hour allocation was invented and the capacity was never tracked. Pulled per audit. Task count is the only signal we can defend without time tracking.

KPI cards

File and function

src/pages/AnalyticsPage.tsx -> computeKpis.

On-time rate

Numerator: tasks with columnId === 'done' AND a dueDate AND daysBetween(today, dueDate) >= 0 (not slipped past today). Denominator: tasks with columnId === 'done' AND a dueDate. Result: Math.round((numerator / denominator) * 100). When denominator is 0 (no closed tasks in window), default to 100. Foot text reads: 'X closed in window · Y on time'.

Honest heuristic note: the data model has no completedAt field, so we can't tell when a card actually moved to Done. We treat 'still on or before due date' as on-time. Imperfect, but defensible. If the portal adds a completedAt field later, swap the heuristic for the real comparison.

Blocked now

Open tasks (isOpen === true) where columnId === 'blocked' OR severity === 'critical'. Foot text reads the oldest blocker's age via humanAge() (uses createdAt sorted ascending; falls back to em-less '-' when no createdAt). Empty state: 'Nothing stuck'.

Due next 7 days

Open tasks with a dueDate where daysBetween(today, dueDate) is in [0, 7]. Foot text reads either ' already overdue' (open tasks with dueDate < today) or 'No overdue in scope'.

Clients flagged

Clients (not tasks) where status === 'fire' OR status === 'risk'. Foot text reads the first three flagged client names joined by ' · ', falling back to 'All clients on track'.

What was pulled

1. Delta chips: every KPI card used to render a chip like '+3% vs prior'. The number was computed from hardcoded tiers (onTimePct > 80 ? 3 : ...), not a real week-over-week comparison. There's no historical snapshot to compare against. Pulled.

2. Sparklines: each KPI card had a 12-point curve drawn from seedHistory(), a deterministic pseudo-random generator. No historical data behind it. Pulled.

3. 'Over capacity' KPI: a fifth card showed % of team over 100% load, computed from a fixed 4h/task allocation against a hardcoded 40h week. Both numerator and denominator were invented. Pulled. Add it back when real time tracking lands.

Capacity / load math

File and function

src/utils/capacity.ts -> loadFor / effectiveCapFor / zoneFor.

loadFor(memberId, dateISO, tasks)

Sum of (task.slots ?? 1) for every task where:

- t.assigneeId === memberId (primary owner only; assigneeIds[] co-owners do not contribute)

- t.dueDate === dateISO (anchored to the due date, not start)

- !t.archived

- t.columnId !== 'done'

Returns 0 when nothing matches. CapacityTask is the union of Task and OpsTask. Both contribute to the same daily load.

effectiveCapFor(memberId, dateISO, members, overrides)

Returns { soft, max } for a (member, date) pair using a three-level fallback:

1. Per-day MemberDayOverride for (memberId, dateISO) -> use those caps.

2. Else member.capSoft / member.capMax if set on the Member record.

3. Else DEFAULT_CAP_SOFT (6) / DEFAULT_CAP_MAX (8).

zoneFor(load, caps)

Color zone classification:

- load <= caps.soft -> 'green'

- load <= caps.max -> 'amber'

- load > caps.max -> 'red'

Amber is the passive 'stretching' signal; red is the booking-flow warning trigger.

nextAvailableDate(...)

Booking-flow helper. Walks forward from fromISO skipping weekends, finds the first weekday where adding `slots` to that date's predicted load (excluding the current task's contribution if excludeTaskId is set) keeps load <= soft cap. Returns null after searchDays (default 14) if nothing fits.

Drill-downs

On-time drill

buildOnTimeDrill: list of closed tasks with dueDate, sorted late-first (overdue before on-time), then by due date descending. Status pill is 'On time' or 'd late'.

Blocked drill

buildBlockedDrill: open tasks where columnId === 'blocked' OR severity === 'critical'. Sort oldest-first using createdAt ascending so week-old blockers sit above this-morning ones. Pill is 'Critical' (severity) or 'Blocked' (column), tone 'late'.

Deadlines drill

buildDeadlinesDrill: open tasks with dueDate. Two groups: overdues first (delta < 0), then upcoming (delta in [0, 7]). Each group sorted by due date ascending. Pills: 'd overdue' (late tone), 'Today' (late), 'Tomorrow' (soon), 'In d' (soon).

Clients drill

buildClientsDrill: clients with status fire or risk. Sort fire before risk, then by name. Each row carries the open task count for that client.

Member drill

Open tasks (isOpen === true) where t.assigneeId === memberId. Sorted by due date ascending; tasks without a due date land at the bottom.

Edge cases

Empty workspace. Zero tasks: every KPI shows 0 with a friendly empty foot text ('Nothing stuck' / 'No overdue in scope' / 'All clients on track'). On-time rate defaults to 100 because dividing by zero without a

default would render NaN.

All tasks unassigned. Workload bar renders every member at 0 wip. The KPIs still count the tasks, so the dashboard shows real numbers; the people view shows nobody is loaded. This is the intentional split: KPIs ask 'what's happening', Workload asks 'who's drowning'.

Member archived or deleted. Tasks owned by a removed member are skipped by the workload loop (the row lookup misses). They still count in KPIs. If you want to surface this as 'orphaned' work, that's a v2 follow-up; v1 just hides it from the people view.

Multi-owner tasks. A task with assigneeIds[] (co-owners) where assigneeId is also set: only the primary assigneeId absorbs the wip / load. Co-owners see nothing on Workload or the heatmap for this task.

Tasks without dueDate. Counted in workload (open is open) but excluded from deadline-based KPIs (Due next 7, Overdue, On-time). Drill-downs that key off dueDate skip them too.

Date window includes lookback. The pills read 'Overdue + next 7 days', 'Overdue + next 30', 'Overdue + next 90'. The label is honest: the range is [today - 14, today + N]. The two-week lookback exists so recent overdues can still register in the overdue counter; the label calls it out so it's not a hidden behaviour.

Filters cascade. applyAnalyticsFilters runs before any KPI compute. So the Anyone / All projects / date window pills shrink the input set. Reset filters returns the page to defaults: assigneeId null, serviceId null, dateWindow '30d'.

Out of scope for this port

Deliberately deferred. Add later only when the data layer supports honest computation.

- Real on-time rate via a completedAt timestamp on the task. Today we compare today vs dueDate as a proxy. Add the field and switch the comparison when the dev build does the same.
- Time tracking and a real Over-capacity KPI. Until cards record actual hours, the percent-of-capacity card is dishonest.
- Weekly snapshots for delta chips and sparklines. Both came back as fake data on the same surface that drives decisions; we pulled them. Bring back when the snapshot store lands.
- Auto-drill into 'Unassigned' as a pseudo-row in Team workload. This is a real follow-up if managers using the unassigned-as-template trick start losing track of the pile, but not v1.
- Holidays affecting the date window math. The Holidays catalog exists, but the date-window math doesn't currently consult it.

Implementation checklist

Tick each step as you ship. Severity tags (HIGH / MED) flag what blocks the page being correct vs. what's polish.

- [ ] HIGH Update applyAnalyticsFilters: drop archived first, then apply assigneeId / serviceId / dateWindow filters. The date-window range is [today - 14, today + N], with N = 7 / 30 / 90 / unlimited.
- [ ] HIGH Update buildWorkload: skip !isOpen, skip !t.assigneeId, skip orphan assignees. wip = count, sharePct = round(wip / max(1, maxWip) * 100). Sort wip descending. Multi-owner assigneeIds[] do NOT contribute.
- [ ] HIGH Update On-time rate: closed (done + dueDate) numerator = on-time-or-not-yet-late. Denominator = all closed in window. Default 100 when denominator is 0.
- [ ] HIGH Update Blocked now: open tasks where columnId === 'blocked' OR severity === 'critical'. Foot text uses oldest blocker's createdAt-derived age.
- [ ] HIGH Update Due next 7 days: open + dueDate + delta in [0, 7]. Foot text reports overdue count when present.
- [ ] HIGH Update Clients flagged: clients with status fire or risk. Foot lists the first three names; falls back to 'All clients on track'.
- [ ] HIGH Match the loadFor / effectiveCapFor / zoneFor / nextAvailableDate rules in capacity.ts. Multi-owner assigneeIds[] never contribute to load. Slots default to 1 when missing.
- [ ] HIGH Match the five drill-down builders: on-time (late-first), blocked (oldest-first), deadlines (overdues then upcoming), clients (fire before risk), member (open tasks for that assignee, due-asc).
- [ ] HIGH Pull the delta chips. The hardcoded-tier comparison is dishonest without weekly snapshots.
- [ ] HIGH Pull the sparkline curves. seedHistory() is a deterministic pseudo-random source; no historical data backs it.
- [ ] HIGH Pull the Over-capacity KPI card and its drill (buildCapacityDrill). Both numerator and denominator were invented; bring it back only when real time tracking lands.
- [ ] MED Update the date-window pill labels to read 'Overdue + next 7 days' / 'Overdue + next 30 days' / 'Overdue + next 90 days' so the lookback is honest.
- [ ] MED Update the on-time rate foot text to read 'X closed in window · Y on time' so the operator can see both numerator and denominator at a glance.
- [ ] MED Confirm filtered tasks feed every section: KPIs, Upcoming deliverables, Workload, every drill. One source of truth; no surface should re-read data.tasks raw.

Reference files in the dev build

Read these to see how the dev build implements each rule. The portal can mirror the same shape or adapt as needed.

src/pages/AnalyticsPage.tsx
applyAnalyticsFilters, computeKpis, buildWorkload, buildOnTimeDrill, buildBlockedDrill, buildDeadlinesDrill, buildClientsDrill, MemberDrillPanel, isOpen helper. The whole computation surface lives in this one file.

src/utils/capacity.ts
loadFor, effectiveCapFor, zoneFor, nextAvailableDate. The per-day load math used by the heatmap and the booking flow.

src/utils/dateFormat.ts
daysBetween + formatMonthDay used throughout. No external date library.

src/types/flizow.ts
Task / OpsTask / Client / Member / MemberDayOverride shapes. Field names referenced above (assigneeId, assigneeIds, severity, dueDate, columnId, slots, archived, capSoft, capMax) all live here.

src/components/TeamCapacityHeatmap.tsx
How the heatmap consumer reads from loadFor / effectiveCapFor / zoneFor. Useful as a worked example.